

Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра ИИТ

Отчет по лабораторной работе № 3
По дисциплине: «Современный платформы программирования»
Специальность: «Программная инженерия»

Выполнил:
О.А. Козюта
Студент группы ПО-13
Проверил:
А.Д. Кулик
Преп.-стажёр кафедры ИИТ

Брест 2026

Цель работы: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python

- Прочитать задания, взятые из каждой группы, соответствующей одному из трех основных типов паттернов;
- Определить паттерн проектирования, который может использоваться при реализации задания. Пояснить свой выбор;
- Реализовать фрагмент программной системы, используя выбранный паттерн. Реализовать все необходимые дополнительные классы.

Первая группа заданий (порождающий паттерн)

8) Торговый автомат с возможностью выдачи любого выбранного товара (шоколадные батончики, чипсы, пакетированные соки и т.д.)

Код:

```
from abc import ABC, abstractmethod
```

```
# ----- Продукты -----
```

```
class Product(ABC):
```

```
    @abstractmethod
```

```
    def get_name(self) -> str:
```

```
        pass
```

```
    @abstractmethod
```

```
    def get_price(self) -> float:
```

```
        pass
```

```
class ChocolateBar(Product):
```

```
    def __init__(self, name: str, price: float):
```

```
        self._name = name
```

```
        self._price = price
```

```
    def get_name(self) -> str:
```

```
        return f"Шоколадный батончик '{self._name}'"
```

```
    def get_price(self) -> float:
```

```
        return self._price
```

```
class Chips(Product):
```

```
    def __init__(self, flavor: str, price: float):
```

```
        self._flavor = flavor
```

```
        self._price = price
```

```
    def get_name(self) -> str:
```

```
        return f"Чипсы со вкусом '{self._flavor}'"
```

```
    def get_price(self) -> float:
```

```
        return self._price
```

```
class Juice(Product):
```

```
    def __init__(self, fruit: str, volume_ml: int, price: float):
```

```
        self._fruit = fruit
```

```
        self._volume = volume_ml
```

```

    self._price = price

def get_name(self) -> str:
    return f'Сок '{self._fruit}' '{self._volume}' мл"

def get_price(self) -> float:
    return self._price

# ----- Создатель (Торговый автомат) -----
class VendingMachine(ABC):
    @abstractmethod
    def create_product(self, product_type: str, **kwargs) -> Product:
        """Фабричный метод для создания товара"""
        pass

    def dispense_product(self, product_type: str, **kwargs) -> None:
        """Метод для выдачи товара"""
        product = self.create_product(product_type, **kwargs)
        print(f'Автомат выдаёт: {product.get_name()} за {product.get_price()} руб.')

# Конкретный создатель
class SnackVendingMachine(VendingMachine):
    def create_product(self, product_type: str, **kwargs) -> Product:
        if product_type == "chocolate":
            return ChocolateBar(kwargs.get("name", "Unknown"), kwargs.get("price", 0.0))
        elif product_type == "chips":
            return Chips(kwargs.get("flavor", "Unknown"), kwargs.get("price", 0.0))
        elif product_type == "juice":
            return Juice(kwargs.get("fruit", "Unknown"), kwargs.get("volume", 0), kwargs.get("price",
0.0))
        else:
            raise ValueError(f"Неизвестный тип товара: {product_type}")

# ----- Пример использования -----
if __name__ == "__main__":
    machine = SnackVendingMachine()
    machine.dispense_product("chocolate", name="Mars", price=75.0)
    machine.dispense_product("chips", flavor="Сметана и лук", price=60.0)
    machine.dispense_product("juice", fruit="Апельсин", volume=330, price=100.0)
    Автомат выдаёт: Шоколадный батончик 'Mars' за 75.0 руб.
    Автомат выдаёт: Чипсы со вкусом 'Сметана и лук' за 60.0 руб.
    Автомат выдаёт: Сок 'Апельсин' 330 мл за 100.0 руб.

```

Вторая группа заданий (структурный паттерн)

8) ДУ телевизора. Реализовать иерархию телевизоров для конкретных производителей и иерархию средств дистанционного управления. Телевизоры должны иметь присущие им атрибуты и функции. ДУ имеет набор функций для изменения текущего канала, увеличения/уменьшения громкости, включения/выключения телевизора и т.д. Эти функции должны отличаться для различных устройств ДУ.

Код:

```
from abc import ABC, abstractmethod
from typing import List, Optional
import os

# ----- Модель файла (получатель) -----
class TextFile:
    def __init__(self, path: str):
        self.path = path
        self._content: Optional[str] = None

    def read(self) -> str:
        with open(self.path, 'r', encoding='utf-8') as f:
            self._content = f.read()
        return self._content

    def write(self, new_content: str) -> None:
        with open(self.path, 'w', encoding='utf-8') as f:
            f.write(new_content)
        self._content = new_content

    def get_content(self) -> Optional[str]:
        return self._content

# ----- Базовый класс команды -----
class Command(ABC):
    @abstractmethod
    def execute(self) -> None:
        pass

    @abstractmethod
    def undo(self) -> None:
        pass

# ----- Конкретные команды -----
class ReadFileCommand(Command):
    def __init__(self, file: TextFile):
        self._file = file
        self._result: Optional[str] = None

    def execute(self) -> None:
        self._result = self._file.read()
        print(f"Прочитан файл {self._file.path}: {self._result[:50]}...")

    def undo(self) -> None:
        # Чтение не изменяет состояние, отмена не требуется
        print(f"Отмена чтения файла {self._file.path} не нужна")

class ModifyFileCommand(Command):
    def __init__(self, file: TextFile, new_content: str):
```

```

self._file = file
self._new_content = new_content
self._old_content: Optional[str] = None

def execute(self) -> None:
    # Сохраняем старое содержимое перед изменением
    self._old_content = self._file.get_content()
    if self._old_content is None:
        # Если файл ещё не читали, прочитаем его
        self._old_content = self._file.read()
    self._file.write(self._new_content)
    print(f"Файл {self._file.path} изменён")

def undo(self) -> None:
    if self._old_content is not None:
        self._file.write(self._old_content)
        print(f"Изменение файла {self._file.path} отменено (восстановлено предыдущее содержимое)")
    else:
        print("Нечего отменять")

# Макрокоманда для последовательного выполнения нескольких операций
class MacroCommand(Command):
    def __init__(self, commands: List[Command]):
        self._commands = commands

    def execute(self) -> None:
        print("Выполнение сложной операции (последовательность команд):")
        for cmd in self._commands:
            cmd.execute()

    def undo(self) -> None:
        print("Отмена сложной операции (отмена всех подкоманд в обратном порядке):")
        for cmd in reversed(self._commands):
            cmd.undo()

# ----- Управляющий класс (Invoker) с историей команд -----
class FileCommandProcessor:
    def __init__(self):
        self._history: List[Command] = []

    def execute_command(self, command: Command) -> None:
        command.execute()
        self._history.append(command)

    def undo_last(self) -> None:
        if self._history:
            cmd = self._history.pop()
            cmd.undo()
        else:

```

```

        print("Нет операций для отмены")

    def undo_all(self) -> None:
        while self._history:
            self.undo_last()

# ----- Пример использования -----
if __name__ == "__main__":
    # Создадим тестовые файлы
    with open("test1.txt", "w", encoding="utf-8") as f:
        f.write("Привет, мир!")
    with open("test2.txt", "w", encoding="utf-8") as f:
        f.write("Старое содержимое")

    file1 = TextFile("test1.txt")
    file2 = TextFile("test2.txt")

    processor = FileCommandProcessor()

    # Одиночные операции
    read_cmd = ReadFileCommand(file1)
    modify_cmd1 = ModifyFileCommand(file2, "Новое содержимое файла 2")

    processor.execute_command(read_cmd)
    processor.execute_command(modify_cmd1)

    print("\n--- Проверка содержимого file2 после изменения ---")
    print(file2.read())

    # Отмена последней операции (изменения)
    processor.undo_last()
    print("\n--- После отмены изменения file2 ---")
    print(file2.read())

    # Сложная операция: последовательное изменение двух файлов
    modify_cmd2 = ModifyFileCommand(file1, "Мир, привет!")
    modify_cmd3 = ModifyFileCommand(file2, "Ещё одна правка")
    macro = MacroCommand([modify_cmd2, modify_cmd3])

    processor.execute_command(macro)
    print("\n--- После выполнения сложной операции ---")
    print(f"file1: {file1.read()}")
    print(f"file2: {file2.read()}")

    # Отмена сложной операции
    processor.undo_last()
    print("\n--- После отмены сложной операции ---")
    print(f"file1: {file1.read()}")
    print(f"file2: {file2.read()}")

```

```
# Очистка тестовых файлов
os.remove("test1.txt")
os.remove("test2.txt")
```

Вывод:

```
Файл test2.txt изменён

--- После выполнения сложной операции ---
file1: Мир, привет!
file2: Ещё одна правка
Отмена сложной операции (отмена всех подкоманд в обратном порядке):
Изменение файла test2.txt отменено (восстановлено предыдущее содержимое)
Изменение файла test1.txt отменено (восстановлено предыдущее содержимое)

--- После отмены сложной операции ---
file1: Привет, мир!
file2: Старое содержимое
```

Вывод: были приобретены навыки применения паттернов проектирования при решении практических задач с использованием языка Python.